Clases Abstractas, Interfaces y Excepciones

Ing. Andrés Eduardo Paternina Ariza Esp.
Universidad del Magdalena
apaternina@unimagdalena.edu.co

Enunciado

Una tienda en línea vende **productos físicos**. Cada producto tiene un **nombre, un precio y una cantidad en stock.**

La tienda quiere ampliar su catálogo incluyendo **productos digitales**, que además del nombre, precio y stock, cuentan con atributos propios.

Producto físico

- Nombre
- Precio
- Cantidad en stock
- Precio final = precio + envío de $10.000

Producto digital

- Nombre, precio y stock
- URL de descarga
- Tipo de licencia (personal o comercial)
- Precio final = precio con 10% de descuento

Nuevos productos

- La tienda seguirá agregando tipos (ej.: suscripciones)
- Cada nuevo tipo define su propio precio final
- Sin modificar las clases existentes

No se pueden registrar **productos genéricos**, solo físicos o digitales. Todos deben poder **calcular su precio final**, pero cada tipo lo hace **a su manera**.

¿Qué es una clase abstracta?

Una clase abstracta es una clase que **no se puede instanciar directamente**: existe únicamente para ser **heredada**.

Estructura común

Define atributos y métodos concretos que comparten todas sus clases hijas.

Métodos obligatorios

Obliga a las clases hijas a implementar los métodos que ella declara pero no define.

Métodos concretos y métodos abstractos

```
abstract class Producto
{
    private string _nombre;

    public string Nombre { get => _nombre; private set => _nombre = value; }
    protected Producto(string nombre) { Nombre = nombre; }

    public abstract double CalcularPrecioFinal();
    public void MostrarInformacion()
    {
        Console.WriteLine($"Producto: {Nombre}");
    }
}
```

Clases hijas: herencia

```
class ProductoFisico : Producto
{
    private double _precio;
    // Property Precio
    public ProductoFisico(string nombre,
        double precio) : base(nombre)
    {
        Precio = precio;
    }
    public override double CalcularPrecioFinal()
    {
        return _precio + 10000;
    }
}
```

```
class ProductoDigital : Producto
{
    private double _precio;
    // Property Precio
    public ProductoDigital(string nombre,
        double precio) : base(nombre)
    {
        Precio = precio;
    }
    public override double CalcularPrecioFinal()
    {
         return _precio * 0.9;
    }
}
```

¿Cuándo usar una clase abstracta?

Base común

- Varias clases comparten atributos y métodos
- El código común se escribe una sola vez
- Ej.: Nombre y MostrarInformacion()

No se instancia

- El concepto es demasiado general
- Un “producto” a secas no existe
- Solo se crean objetos de las clases hijas

Cada hija a su manera

- Se sabe qué hacer, no cómo hacerlo
- El método abstracto obliga a implementarlo
- Ej.: CalcularPrecioFinal()

Enunciado

La tienda en línea quiere aceptar varios **medios de pago**: tarjeta de crédito, Nequi y efectivo contra entrega.

Cada medio funciona distinto y **no comparten atributos**, pero todos deben poder **pagar** y **generar un comprobante**.

Tarjeta de crédito

- Número de tarjeta
- Número de cuotas
- Pagar = cobra el valor en cuotas

Nequi

- Número de celular
- Pagar = envía una notificación al celular para aprobar

Efectivo contra entrega

- Dirección de entrega
- Pagar = el repartidor recibe el dinero

Los medios de pago **no son una familia** y no comparten código: solo deben cumplir el mismo **contrato**, los métodos **Pagar()** y **GenerarComprobante()**.

¿Qué es una interfaz?

Una interfaz es un **contrato**: solo declara **qué métodos y propiedades** debe tener una clase, sin implementarlos.

También puede tener propiedades

Nombre { get; }. Cada clase que la implementa debe incluirlas.

Cualquier clase la adopta

Las clases que la implementan deben escribir todos sus métodos, aunque no sean de la misma familia.

Estructura de una interfaz

```
interface IMedioPago
{
    void Pagar(double monto);
    string GenerarComprobante();
}
```

```
class Nequi : IMedioPago
{
    public void Pagar(double monto)
    {
        Console.WriteLine("Pago con Nequi");
    }
    public string GenerarComprobante()
    {
        return "Comprobante Nequi";
    }
}
```

Implementación: TarjetaCredito

```
class TarjetaCredito : IMedioPago
{
    private int _cuotas;
    // Property
    public int Cuotas { get => _cuotas; private set => _cuotas = value; }
    public TarjetaCredito(int cuotas) { Cuotas = cuotas; }
    public void Pagar(double monto)
    {
        Console.WriteLine($"{Cuotas} cuotas de ${monto / Cuotas}");
    }
    public string GenerarComprobante()
    {
        return $"Tarjeta - {Cuotas} cuotas";
    }
}
```

Implementación: Nequi

```
class Nequi : IMedioPago
{
    private string _celular;
    // Property
    public string Celular { get => _celular; private set => _celular = value; }
    public Nequi(string celular) { Celular = celular; }
    public void Pagar(double monto)
    {
        Console.WriteLine($"Notificación enviada al {Celular}");
    }
    public string GenerarComprobante()
    {
        return $"Nequi - {Celular}";
    }
}
```

Implementación: EfectivoContraEntrega

```
class EfectivoContraEntrega : IMedioPago
{
    private string _direccion;
    // Property
    public string Direccion { get => _direccion; private set => _direccion = value; }
    public EfectivoContraEntrega(string direccion) { Direccion = direccion; }
    public void Pagar(double monto)
    {
        Console.WriteLine($"Cobrar ${monto} en {Direccion}");
    }
    public string GenerarComprobante()
    {
        return $"Contra entrega - {Direccion}";
    }
}
```

COMPARACIÓN

Clase abstracta vs interfaz

Criterio	Clase Abstracta	Interfaz
¿Tiene implementación?	Sí — métodos concretos y abstractos	No — solo firmas de métodos
¿Atributos?	Sí, mantiene estado	No, sin estado
¿Herencia múltiple?	No: se hereda de una sola	Sí: una clase firma varias
¿Cuándo usarla?	Familia con código compartido	Habilidad que cualquier clase adopta

Clase abstracta = familia con código compartido. **Interfaz** = habilidad que cualquier clase adopta.

¿Se puede implementar con private set?

```
interface IMedioPago
{
    string Titular { get; set; }
}
class Nequi : IMedioPago
{
    // Error: el set debe ser público
    public string Titular { get; private set; }
}
```

```
interface IMedioPago
{
    string Titular { get; }
}
class Nequi : IMedioPago
{
public string Titular { get; private set; }
}
```

No. El contrato **{ get; set; }** exige que cualquiera que use la interfaz pueda **modificar la propiedad públicamente**. Con **{ get; }** sí se puede.

Enunciado

Un parqueadero registra el ingreso de vehículos mediante **tiquetes** que contienen la **placa**, las **horas de permanencia** y la **tarifa asignada**.

Cada tiquete debe poder **calcular el valor a pagar** según la tarifa que tenga asignada.

Visitante ocasional

- Paga $4.000 por cada hora de permanencia

Cliente con convenio

- Las primeras 2 horas son gratis
- Paga $2.500 por cada hora adicional

Nuevas modalidades

- El parqueadero agregará otras tarifas
- Sin alterar los tiquetes existentes

Todas las tarifas cumplen el mismo **contrato**: calcular el valor según las horas. El tiquete solo usa ese contrato, sin saber qué tarifa tiene.

Resolución: parqueadero

```
interface ITarifa
{
    double Calcular(int horas);
}
class TarifaOcasional : ITarifa
{
    public double Calcular(int horas)
    {
        return horas * 4000;
    }
}
class TarifaConvenio : ITarifa
{
    public double Calcular(int horas)
    {
        return Math.Max(0, horas - 2) * 2500;
    }
}
```

```
class Tiquete
{
    public string Placa { get; private set; }
    public int Horas { get; private set; }
    public ITarifa Tarifa { get; private set; }

    public Tiquete(string placa, int horas, ITarifa tarifa)
    {
        Placa = placa;
        Horas = horas;
        Tarifa = tarifa;
    }

    public double CalcularValor()
    {
        return Tarifa.Calcular(Horas);
    }

    public void ActualizarTarifa(ITarifa nuevaTarifa)
    {
        Tarifa = nuevaTarifa;
    }
}
```

Uso: clase Program

```
class Program
{
    static void Main(string[] args)
    {
        Tiquete t1 = new Tiquete("ABC123", 5, new TarifaOcasional());
        Tiquete t2 = new Tiquete("XYZ789", 5, new TarifaConvenio());

        Console.WriteLine($"{t1.Placa}: ${t1.CalcularValor()}");  // ABC123: $20000
        Console.WriteLine($"{t2.Placa}: ${t2.CalcularValor()}");  // XYZ789: $7500


        t1.ActualizarTarifa(new TarifaConvenio());
        Console.WriteLine($"{t1.Placa}: ${t1.CalcularValor()}");  // ABC123: $7500
    }
}
```

Enunciado

Los **medios de pago** de la tienda reciben datos que pueden llegar mal: cuotas en cero, un celular vacío o un monto negativo.

Cada clase debe **validar** esos datos y **lanzar su propia excepción** en su propiedad o en Pagar().

TarjetaCredito

- Propiedad Cuotas
- Si cuotas ≤ 0, el set lanza CuotasInvalidasException

Nequi

- Propiedad Celular
- Si llega vacío o null, el set lanza CelularInvalidoException

EfectivoContraEntrega

- Método Pagar(monto)
- Si el monto ≤ 0, lanza MontoInvalidoException

El programa hace cada pago dentro de **try / catch**, muestra **ex.Message** y **sigue funcionando**.

¿Qué es una excepción?

Una excepción es un **error en tiempo de ejecución** que interrumpe el programa si nadie la **captura**.

Se lanza con throw

El código detecta un problema y lanza la excepción, ej.: throw new ArgumentException("Monto inválido").

Se captura con try / catch

El bloque catch atrapa el error, muestra un mensaje y el programa sigue funcionando.

Capturar errores con try / catch

```
try
{
    // Código que puede fallar
}
catch (Exception ex)
{
    // Qué hacer si falla
}
finally
{
    // Se ejecuta siempre
}
```

```
try
{
    if (monto <= 0)
        throw new ArgumentException(
            "Monto inválido");
    medio.Pagar(monto);
}
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);
}
catch (Exception ex)
{
    // Generico
}
```

EXCEPCIONES

Principales excepciones en C#

Excepción	Cuándo ocurre	Ejemplo
NullReferenceException	Se usa un objeto que es null	celular.Length con celular = null
DivideByZeroException	Se divide un entero entre 0	total / cuotas con cuotas = 0
FormatException	Un texto no tiene el formato esperado	int.Parse("abc")
IndexOutOfRangeException	Se accede a una posición que no existe	productos[10] en un arreglo de 5
ArgumentException	Un método recibe un valor inválido	Pagar(-500)
InvalidOperationException	La operación no es válida ahora	Pagar un pedido ya pagado

Captura primero las excepciones **específicas** y deja **Exception** al final como respaldo general.

Crear excepciones personalizadas

```
class MontoInvalidoException : Exception
{
    public MontoInvalidoException(
        string mensaje) : base(mensaje) { }
}
class CuotasInvalidasException : Exception
{
    public CuotasInvalidasException(
        string mensaje) : base(mensaje) { }
}
```

```
public void Pagar(double monto)
{
    if (monto <= 0)
        throw new MontoInvalidoException(
            "El monto debe ser mayor a 0");
}
try { efectivo.Pagar(-500); }
catch (MontoInvalidoException ex)
{
    Console.WriteLine(ex.Message);
}
```

Taller práctico: centro veterinario

Una clínica veterinaria ha decidido desarrollar un sistema que permita **gestionar los servicios** prestados a sus pacientes y **calcular los valores** que deben pagar sus propietarios.

Todas las atenciones conservan información que permite **identificarlas y describirlas**, aunque cada una **determina su costo** de acuerdo con sus propias características.

Consulta veterinaria

- Valor de $55.000
- Se agregan $20.000 si la atención es por urgencias

Procedimiento clínico

- Valor inicial de $90.000
- Aumenta $15.000 por cada unidad de insumo utilizada

Nuevas atenciones

- La clínica incorporará otros tipos en el futuro
- No registra servicios sin una modalidad específica

Taller: registro y cobertura

Cada paciente cuenta con un **registro de atención** que contiene su **identificación, nombre y especie**, junto con las atenciones que ha recibido durante su visita.

Un paciente puede requerir **varios servicios** dentro del mismo registro, y el **valor acumulado** debe reflejar los costos individuales de todas las atenciones.

Propietario particular

- Asume la totalidad de los servicios recibidos

Convenio empresarial

- Descuento del 20% sobre el valor acumulado
- El descuento no puede superar $60.000

Nuevas coberturas

- La administración agregará otras modalidades
- Sin modificar los registros de atención existentes

Cada registro conserva la **modalidad de cobertura** del paciente, y esta determina el **valor definitivo** que debe pagar su propietario.

Taller: cierre y errores

Una vez finalizada la visita, el registro debe **cerrarse** para dejar constancia de que todos los servicios fueron prestados.

Reglas del registro

- No se puede cerrar un registro que no contenga atenciones
- No se pueden incorporar servicios después del cierre

Errores específicos

- Procedimientos con cantidades de insumos incorrectas
- Valores que no corresponden a una atención válida
- Operaciones que el estado del registro no permite

Consulta del registro

- Información del paciente
- Atenciones con sus valores
- Costo acumulado y cobertura aplicada
- Beneficio obtenido y total a pagar

Cada error debe identificar el problema, sin que una **operación fallida** interrumpa el procesamiento de los **demás pacientes**.

Taller: escenarios de verificación

El sistema debe demostrar su funcionamiento con estos **pacientes** y **casos de error**:

Paciente Luna

- Gata con una consulta de urgencias
- Procedimiento clínico con 2 unidades de insumo
- Propietario con convenio empresarial

Paciente Rocky

- Perro con una consulta veterinaria ordinaria
- Propietario sin ningún convenio

Casos de error

- Registrar un procedimiento con cantidades inválidas
- Cerrar un registro sin atenciones
- Agregar un servicio a un registro cerrado

Desarrolla la solución en **C#** aplicando los principios de POO del curso, de modo que se puedan incorporar **nuevas modalidades de atención y cobertura** sin alterar innecesariamente las clases existentes.